



Study Programme

42484 Information Systems (BSc)

Department of Natural Sciences and Computer Engineering

**RIGA NORDIC UNIVERSITY
(RNU)**

**VAKSINA UZ — Cross-Platform Vaccination
Management System**

BACHELOR'S THESIS

Student: Akbarali Holmatov

Supervisor: Dr.sc.ing., Prof. Jelena Caiko

Riga 2026

Anotācija

Bakalaura darbs ir veltīts mobilās un tīmekļa lietojumprogrammas “VAKSINA UZ” izstrādei — starpplatformu vakcinācijas pārvaldības sistēmai, kas paredzēta Uzbekistānas primārās veselības aprūpes iestādēm un bērnu vecākiem. Tēma ir aktuāla, jo bērnu profilaktiskā vakcinācija joprojām lielā mērā tiek pārvaldīta, izmantojot papīra kartes, kas izraisa nokavētas devas, nepilnīgus pārklājuma datus un vāju saziņu starp poliklīniku un ģimeni.

Darbs sastāv no sešām nodaļām. 1. nodaļā apskatīta zinātniskā literatūra par imunizācijas informācijas sistēmām, atgādinājumu mehānismiem un starpplatformu izstrādes ietvariem. 2. nodaļā analizēta problēmas joma, lietotāju lomas, funkcionālās un nefunkcionālās prasības, kā arī sistēmas arhitektūra. 3. nodaļā aprakstīta praktiskā izstrāde: datu modelis, Flutter klienta slānis, Firebase aizmugursistēma un automātiskā vakcinācijas grafika aprēķināšana. 4. nodaļā novērtēta sistēmas veikspēja, lietojamība un grafika algoritma pareizība. 5. nodaļā apspriesti ierobežojumi un turpmākā darba virzieni. 6. nodaļā sniegti secinājumi.

Pētījuma metodoloģiskais pamats ietver zinātniskās literatūras analīzi, prasību inženieriju, prototipēšanu, programmatūras izstrādi un eksperimentālu veikspējas mērīšanu. Galvenais rezultāts ir pilnībā funkcionējoša starpplatformu sistēma, kas automātiski aprēķina Uzbekistānas Nacionālā imunizācijas kalendāra (2025) grafiku, atzīmē nokavētās un gaidāmās devas un sniedz reāllaika atgādinājumus gan medicīnas personālam, gan vecākiem.

Bakalaura darbs izstrādāts 45 lappusēs un ietver 9 tabulas, 11 attēlus, 2 algoritmus, 6 programmu kodu izdrukas, 2 pielikumus, kā arī literatūras sarakstu ar 21 avotu.

Atslēgvārdi: vakcinācija, imunizācijas informācijas sistēma, starpplatformu izstrāde, Flutter, Firebase, mobilā veselība, atgādinājumu sistēma.

Abstract

This Bachelor Paper is devoted to the design and implementation of “VAKSINA UZ”, a cross-platform vaccination management system intended for primary health-care facilities and parents of children in Uzbekistan. The topic is relevant because childhood preventive immunization in many districts is still managed through paper-based vaccination cards, which leads to missed doses, incomplete coverage statistics, and weak communication between the polyclinic and the family. A single digital system that automatically computes each child’s vaccination schedule and reminds both clinicians and parents can directly improve timeliness and coverage.

The Bachelor Paper consists of six chapters. Chapter 1 reviews the scientific literature on immunization information systems, reminder mechanisms, and cross-platform application frameworks, and establishes the theoretical framework for the work. Chapter 2 analyses the problem domain, defines the user roles (hospital staff and parents), elicits functional and non-functional requirements, and presents the system architecture. Chapter 3 contains the practical research: the Firestore data model, the Flutter client layer, the Firebase back-end services, and the automatic schedule-calculation engine based on the Uzbekistan National Immunization Calendar of 2025. Chapter 4 evaluates the system’s performance, usability, and the correctness of the schedule algorithm against a manually computed baseline. Chapter 5 discusses limitations and future work, and Chapter 6 states the conclusions.

The methodological basis of the study includes the analysis of scientific literature, requirements engineering, iterative prototyping, software construction, and experimental measurement. The applied methods are comparative analysis, system modelling, prototype-based experimentation, and descriptive statistical evaluation of test results. The main result of the work is a fully functional cross-platform system that automatically derives the immunization schedule for each registered child, flags overdue, due, and upcoming doses, computes coverage statistics for the clinic, and delivers real-time reminders to both medical personnel and parents through a shared Firebase back-end.

The Bachelor Paper is written in 45 pages and contains 9 tables, 11 figures, 2 algorithms, 6 listings, 2 annexes, and a list of references with 21 sources.

Keywords: vaccination, immunization information system, cross-platform development, Flutter, Firebase, mobile health, reminder system.

Key words / Atslēgvārdi

English	Latviešu / O'zbekcha
Vaccination	Vakcinācija
Immunization information system	Imunizācijas informācijas sistēma
Cross-platform development	Starpplatformu izstrāde
Flutter framework	Flutter ietvars
Firebase / cloud back-end	Firebase mākoņa aizmugursistēma
Mobile health (mHealth)	Mobilā veselība
Reminder / notification system	Atgādinājumu sistēma
Vaccination coverage	Vakcinācijas pārklājums

Contents

Anotācija	1
Abstract	2
Key words / Atslēgvārdi	3
List of Figures	6
List of Tables	7
Listings	8
List of Algorithms	9
Introduction	10
1 Literature Review and Theoretical Background	13
1.1 Core Concepts and Definitions	13
1.2 Related Work: Themes, Methods, and Evidence	13
1.3 Theoretical Framework	14
1.4 Illustrative Code Snippet	16
1.5 Research Motivation and Direction	16
1.6 Chapter Summary	17
2 System Analysis and Architecture	18
2.1 General Characteristics of the Problem Domain	18
2.2 User Roles and Use Cases	18
2.3 Functional and Non-Functional Requirements	19
2.4 System Architecture	20
2.5 Data Model	20
2.6 Context, Constraints, and Risks	21
2.7 Sub-Conclusions (for Chapter 2)	21
3 Practical Implementation of the System	22
3.1 Implementation Strategy and Technology Stack	22
3.2 Cloud Data Layer	22
3.3 Service Layer	23
3.4 The Scheduling Engine	24

3.5	Notification Subsystem	26
3.6	User Interface	26
3.7	Sub-Conclusions (for Chapter 3)	27
4	Evaluation of the System	30
4.1	Design of the Evaluation	30
4.2	Correctness of the Scheduling Engine	30
4.3	Performance	31
4.4	Heuristic Usability Evaluation	32
4.5	Sub-Conclusions (for Chapter 4)	32
5	Discussion, Limitations, and Future Work	34
5.1	Interpretation of the Results	34
5.2	Limitations	34
5.3	Future Work	35
5.4	Sub-Conclusions (for Chapter 5)	35
6	Conclusions	36
6.1	Confirmation of the Tasks	36
6.2	Validation of the Hypothesis	36
6.3	Achievement of the Aim	37
6.4	Practical and Theoretical Value	37
	References	38
A	Project Structure and Additional Code	40
A.1	Source Tree Overview	40
A.2	Aggregate Statistics Computation	40
B	Expert Evaluation Criteria for Final Thesis Quality	42
	Apiecinājums / Affirmation	43
	Pateicības / Acknowledgments	44

List of Figures

1.1	Conceptual taxonomy of the capabilities a digital immunization system must provide, derived from the literature in chapter 1.	16
2.1	Principal use cases of <i>VAKSINA UZ</i> and their association with the two human actors.	19
2.2	Three-layer architecture of <i>VAKSINA UZ</i> : a cross-platform Flutter client, a Dart service layer, and a Firebase back-end.	20
2.3	Logical data model: the patients collection, its vaccinations sub-collection, and the users collection that carries roles.	21
3.1	Reactive data flow: a write by one client is pushed to all subscribers as a new snapshot.	24
3.2	Screen navigation map for the two roles after authentication.	26
3.3	Authentication and the clinical-staff dashboard.	27
3.4	Patient registration and the colour-coded personal schedule.	28
3.5	Population statistics for staff and the simplified parent view.	29
4.1	Expected versus matched doses per test profile; the two series coincide, indicating full agreement.	31
4.2	Indicative latency of local schedule derivation versus the first cloud read across target platforms.	32

List of Tables

1	Mapping of research tasks to thesis chapters	12
1.1	Comparison of mobile development approaches for this project	14
2.1	Uzbekistan National Immunization Calendar 2025 (as encoded in the system)	18
2.2	Functional requirements (MoSCoW priority)	19
2.3	Representative system metrics of the prototype	20
4.1	Per-profile agreement between the engine and the manual reference schedule.	31
4.2	Heuristic usability inspection of the two role flows.	33
A.1	Principal directories of the <code>lib/</code> source tree.	40
B.1	Expert evaluation criteria for the quality of the final thesis.	42

Listings

1.1	Data type for one entry of the national immunization calendar.	16
3.1	Serialisation contract between the <code>PatientModel</code> and the Firestore document.	22
3.2	Create and real-time read operations in the service layer.	23
3.3	Parent-side query that resolves a caregiver's own children by phone number.	24
3.4	Deterministic per-child schedule derivation (<code>calculateVaccineSchedule</code>).	24
A.1	Population-level statistics derived by reusing the scheduling engine. . . .	40

List of Algorithms

1	Per-child schedule derivation (template form of eqs. (1.1) and (1.2))	15
2	Reminder candidate selection from the derived schedule	26

Introduction

Topicality of the problem

Childhood immunization is one of the most cost-effective public-health interventions in existence, preventing an estimated three to four million deaths worldwide every year. Uzbekistan operates a National Immunization Calendar that schedules more than two dozen vaccine doses against thirteen infectious diseases during the first sixteen years of a child's life. Despite a historically high reported coverage, the day-to-day management of these schedules in many family polyclinics still relies on paper vaccination cards (the so-called *Form 063*) and manual recall by nurses. Paper records are difficult to aggregate into district-level statistics, are easily lost when a family relocates, and provide no automatic mechanism to warn a parent that a dose is approaching or already overdue.

The consequence is a class of avoidable problems: doses administered late or skipped entirely, double registration of the same child in different facilities, and a lack of timely feedback to parents. The COVID-19 period further demonstrated how quickly a health system needs to scale reminders and coverage reporting, and how poorly paper-based workflows cope with that demand. A lightweight digital tool that runs on the phones nurses and parents already own — and that keeps a single shared record in the cloud — can address these problems without requiring expensive infrastructure. This combination of clear practical need and accessible modern technology makes the topic both timely and feasible for a Bachelor-level engineering project.

Aim of the study

The aim of the study is to design and implement a cross-platform vaccination management system that automatically computes the immunization schedule for each child according to the Uzbekistan National Immunization Calendar, and that supports both health-care personnel and parents through a single shared cloud back-end.

Object and subject of the study

The **object** of the study is the digital management of childhood preventive immunization. The **subject** of the study is the architecture, schedule-calculation logic, and cross-platform implementation of a vaccination management information system for primary health care.

Research problem

The central research problem is the absence of an accessible, automated tool that (i) keeps a single shared immunization record for each child, (ii) derives the correct individual schedule and dose status without manual computation, and (iii) delivers timely reminders to both clinicians and parents across the heterogeneous devices they use.

Tasks of the study

To achieve the aim, the following tasks were set; each task corresponds directly to a chapter of the thesis, as summarised in table 1.

1. Review the scientific literature on immunization information systems, reminder mechanisms, and cross-platform frameworks, and establish a theoretical framework.
2. Analyse the problem domain, define user roles, and elicit functional and non-functional requirements; design the system architecture and data model.
3. Implement the system: the data layer, the schedule-calculation engine, the cross-platform client, and the cloud back-end.
4. Evaluate the resulting system in terms of correctness, performance, and usability.
5. Discuss limitations and outline directions for future work.

Hypothesis

If the immunization schedule and dose status are computed automatically from each child’s date of birth and the national calendar, and surfaced through a shared cross-platform application, then the timeliness of scheduling decisions and the completeness of coverage information will improve compared with a paper-based workflow, while the marginal cost per facility remains negligible.

Research methods

The work applies: analysis of scientific literature and official documents; requirements engineering; system and data modelling (UML-style use-case and component diagrams, entity diagrams); iterative prototyping and software construction in Dart/Flutter and Firebase; and experimental, descriptive-statistical evaluation of the prototype (algorithm correctness against a manual baseline, latency measurement, and a small heuristic usability assessment).

Approbation of the study

The prototype was demonstrated to the supervisor during development reviews, and the schedule-calculation engine was validated against manually prepared reference schedules for a set of synthetic patient records. The complete source code of the client application is included as the technical artefact accompanying this thesis.

Table 1. Mapping of research tasks to thesis chapters

Task	Description	Chapter
Task 1	Literature review and theoretical framework	Chapter 1
Task 2	Domain analysis, requirements, architecture	Chapter 2
Task 3	Implementation of the system	Chapter 3
Task 4	Evaluation of correctness, performance, usability	Chapter 4
Task 5	Discussion, limitations, future work	Chapter 5

1. Literature Review and Theoretical Background

1.1. Core Concepts and Definitions

Several concepts recur throughout this thesis and are defined here once for consistent use.

An **Immunization Information System (IIS)** is a confidential, population-based, computerised database that records vaccination doses administered by participating providers within a given geographic area, consolidates an individual’s vaccination history, and produces aggregate coverage data for public-health monitoring [1, 2]. A **vaccination schedule** (or immunization calendar) is the age-indexed list of antigens and doses that a national health authority recommends; in this work the reference schedule is the Uzbekistan National Immunization Calendar of 2025 [9, 7]. A **reminder** is a notification sent to a patient or guardian *before* a dose is due, whereas a **recall** targets a dose that is already overdue [2, 4].

On the engineering side, a **cross-platform framework** is a toolkit that compiles a single codebase to multiple target platforms (Android, iOS, web, desktop). **Flutter** is such a framework, using the Dart language and a widget-based reactive UI model [14, 10]. **Firebase** is a Backend-as-a-Service (BaaS) platform providing managed authentication, a document database (Cloud Firestore), and push messaging [15, 16]. **mHealth** (mobile health) denotes the use of mobile devices to support medical and public-health practice [8].

1.2. Related Work: Themes, Methods, and Evidence

The literature relevant to this thesis falls into three strands: the public-health evidence for digital immunization tools, the technical literature on reminder delivery, and the engineering literature on cross-platform mobile development.

Strand A — Immunization information systems and coverage. A systematic review for the U.S. Community Preventive Services Task Force screened 240 studies and concluded that immunization information systems are effective at increasing vaccination rates, principally because they enable reminder/recall interventions, provider feedback, and accurate coverage assessment [1, 2]. A more recent world-wide review of IIS implementations reports that systems supporting a life-course vaccination record and automated recall are associated with a median increase of roughly six percentage points in vaccination rates, while also noting persistent gaps in interoperability and a near-absence of cost-effectiveness data in low- and middle-income countries [5].

Strand B — Reminder and recall mechanisms. The classic review by Szilagyi and colleagues found that patient reminder systems improved immunization rates in 80% of 41 analysed studies, with absolute increases of five to twenty percentage points and odds ratios around 2.0 for childhood vaccination [3]. A Cochrane review subsequently identified telephone short-message-service (SMS) reminders as the single most effective reminder modality in primary care [4]. For low- and middle-income countries specifically, a 2025 meta-analysis of information-and-communication-technology interventions reported odds ratios of about 2.3 for third-dose pentavalent coverage and 2.6 for full immunization at one year of age, though with high heterogeneity and recurring concerns about data protection and interoperability [6]. The collective evidence motivates a design in which reminders are a first-class feature rather than an afterthought.

Strand C — Cross-platform development. Comparative studies of mobile development approaches distinguish native, hybrid (web-view), and compiled cross-platform frameworks. Flutter, which compiles Dart ahead-of-time to native ARM code and renders its own widgets, is repeatedly reported to approach native performance while preserving a single codebase [10, 11, 12]. Pairing a Flutter client with a managed BaaS such as Firebase is a common pattern for small teams, since it removes the need to operate servers, provides real-time data synchronisation, and offers built-in authentication and messaging [16, 13]. The trade-off, widely noted, is vendor lock-in and limited control over the data layer [13].

Table 1.1 contrasts the three principal mobile-development approaches against the criteria most relevant to a single-developer health-care project.

Table 1.1. Comparison of mobile development approaches for this project

Criterion	Native (per OS)	Hybrid (web-view)	Flutter
Single codebase	No	Yes	Yes
Runtime performance	Highest	Lower	Near-native
UI consistency	Per-OS	Variable	Consistent
Development effort	High ($\times N$)	Medium	Low
Web + desktop reach	Limited	Yes	Yes

When referring to figures, tables, algorithms, and listings, this thesis uses cross-references, e.g. fig. 1.1, table 1.1, algorithm 1, and listing 1.1.

1.3. Theoretical Framework

The work is grounded in two complementary lenses.

(1) The schedule as a deterministic function. The central computational object is the mapping from a child’s date of birth to a set of due dates. Let b be the date of birth and let the national calendar be a finite set of antigens $V = \{(n_i, d_i, g_i)\}$, where n_i is the antigen name, d_i the recommended age in days, and $g_i \in \{\text{all}, \text{female}\}$ an eligibility constraint. The scheduled date of dose i is

$$s_i(b) = b + d_i \quad (\text{in days}), \quad (1.1)$$

and its status at evaluation time t is a function of the signed gap $\Delta_i = s_i(b) - t$:

$$\text{status}_i(t) = \begin{cases} \text{OVERDUE} & \Delta_i < -30, \\ \text{DUE} & -30 \leq \Delta_i < 0, \\ \text{UPCOMING} & 0 \leq \Delta_i \leq 7, \\ \text{SCHEDULED} & \Delta_i > 7. \end{cases} \quad (1.2)$$

This formalisation makes the schedule reproducible, testable, and independent of any manual judgement — the property the hypothesis depends on. Algorithm 1 expresses it procedurally.

Algorithm 1: Per-child schedule derivation (template form of eqs. (1.1) and (1.2))

Input: date of birth b , calendar V , gender g , today t

Output: list of scheduled doses with status

```

1  $S \leftarrow \emptyset$ ;
2 foreach  $(n_i, d_i, g_i) \in V$  do
3   if  $g_i = \text{female}$  and  $g \neq \text{female}$  then
4     continue
5    $s_i \leftarrow b + d_i$  days;
6    $\Delta_i \leftarrow s_i - t$  in days;
7   assign  $\text{status}_i$  per eq. (1.2);
8    $S \leftarrow S \cup \{(n_i, s_i, \text{status}_i)\}$ ;
9 return  $S$ ;
```

(2) The Backend-as-a-Service architectural lens. The system is viewed as a thin reactive client over a shared document store. State is held authoritatively in the cloud; clients subscribe to changes and re-render. This lens (sometimes called *offline-first reactive* or *serverless client-centric*) shapes every later design decision, from the choice of a NoSQL document model to the use of real-time listeners instead of polling [16, 15].

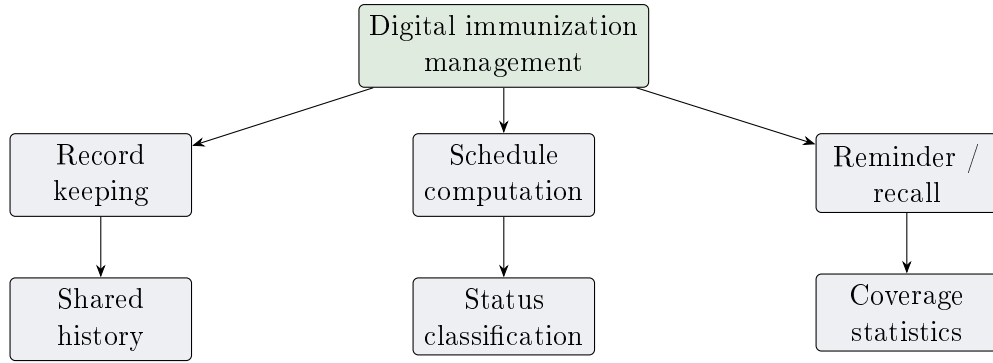


Figure 1.1. Conceptual taxonomy of the capabilities a digital immunization system must provide, derived from the literature in chapter 1.

1.4. Illustrative Code Snippet

The deterministic calendar of eq. (1.1) is represented in code as an immutable list of typed records. Listing 1.1 shows the data type used for a single calendar entry; the full calendar (27 doses) is built from instances of this type.

```

1  class VaccineSchedule {
2      final String name;           // full antigen name
3      final String shortName;      // abbreviation, e.g. "Penta-1"
4      final String description;    // human-readable note
5      final int ageInDays;         // recommended age, in days from birth
6      final String ageLabel;       // e.g. "2 oylik" (2 months)
7      final String protectsFrom;   // disease(s) prevented
8      VaccineSchedule({
9          required this.name, required this.shortName,
10         required this.description, required this.ageInDays,
11         required this.ageLabel, required this.protectsFrom,
12     });
13 }

```

Listing 1.1. Data type for one entry of the national immunization calendar.

1.5. Research Motivation and Direction

Problem significance. The evidence in Strands A and B shows that reminder-capable information systems reliably raise vaccination coverage, yet the same reviews emphasise that low- and middle-income settings lack affordable, interoperable, parent-facing tools [5, 6]. Uzbekistan’s primary care, still largely paper-based, is exactly such a setting.

Identified gap. No widely available tool combines (i) automatic, deterministic schedule derivation from the *national* calendar, (ii) a *shared* record usable by both clinic and parent, and (iii) *cross-platform* reach achievable by a single developer at negligible

marginal cost.

Direction and scope. This thesis addresses the gap by building a Flutter + Firebase system that computes the schedule per eqs. (1.1) and (1.2) and exposes it to two roles. Out of scope are official integration with state registries, cold-chain/stock management, and clinical decision support beyond scheduling.

Success criteria. The schedule engine must reproduce a manually computed reference schedule exactly; common screens must load within an interactive latency budget; and the two user roles must each be able to complete their core task without training.

1.6. Chapter Summary

This chapter fixed the key concepts (chapter 1), contrasted native, hybrid, and Flutter development (table 1.1), surveyed the public-health and technical evidence, and established a two-part theoretical framework: a deterministic schedule function (eqs. (1.1) and (1.2)) and a Backend-as-a-Service architectural lens.

Conclusions (for Chapter 1). The literature confirms that reminder-capable immunization information systems improve coverage, that an affordable parent-facing tool is missing in settings like Uzbekistan, and that a Flutter + Firebase stack is an appropriate, low-cost means of closing that gap. These findings motivate the requirements and architecture developed in chapter 2.

2. System Analysis and Architecture

This chapter analyses the problem domain, derives the requirements, and specifies the architecture and data model of *VAKSINA UZ*. It corresponds to Task 2 of the Introduction.

2.1. General Characteristics of the Problem Domain

The system serves two distinct user groups whose needs differ sharply. **Hospital staff** (a nurse or paediatrician in a family polyclinic) need to register children, record administered doses, see at a glance which children are overdue, and report coverage. **Parents** need a simple, read-mostly view of their own child’s progress and timely reminders of the next dose. A single system must therefore present two role-specific experiences over one shared record.

The clinical content is fixed by the Uzbekistan National Immunization Calendar of 2025, which prescribes 27 doses of 13 distinct vaccines between birth and the age of sixteen. The calendar is summarised in table 2.1; it is the authoritative input to the schedule engine of eq. (1.1).

Table 2.1. Uzbekistan National Immunization Calendar 2025 (as encoded in the system)

Age	Vaccine (dose)	Protects against
0–24 h	Hepatitis B (HepB-1)	Viral hepatitis B
1–5 days	BCG	Tuberculosis
2 months	Penta-1, OPV-1, Rota-1, Pneumo-1	Diphtheria, pertussis, tetanus, HepB, Hib, polio, rotavirus, pneumococcus
3 months	Penta-2, OPV-2, Rota-2, Pneumo-2	As above (2nd dose)
4 months	Penta-3, IPV, Rota-3	As above (3rd dose), polio
9 months	IPV-2, MR-1	Polio, measles, rubella
12 months	MMR-1, Pneumo-3	Measles, mumps, rubella, pneumococcus
16 months	DTP-R, OPV-R	Diphtheria, pertussis, tetanus, polio
6 years	MMR-2, ADS-M	Measles, mumps, rubella, diphtheria, tetanus
9 years	HPV-1 (girls)	Cervical cancer
16 years	ADS-M	Diphtheria, tetanus

2.2. User Roles and Use Cases

Three actors interact with the system: the *Hospital user*, the *Parent*, and the *System* itself (which performs scheduled computation and notification). Figure 2.1 presents the

principal use cases.

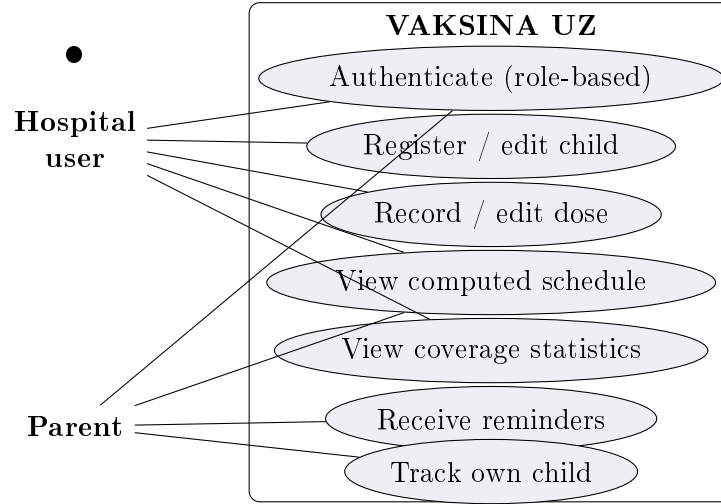


Figure 2.1. Principal use cases of *VAKSINA UZ* and their association with the two human actors.

2.3. Functional and Non-Functional Requirements

Requirements were elicited from the domain analysis and the literature, then prioritised with the MoSCoW method. Table 2.2 lists the functional requirements.

Table 2.2. Functional requirements (MoSCoW priority)

ID	Requirement	Priority
FR-1	The system shall authenticate users and assign a role (hospital / parent).	Must
FR-2	A hospital user shall create, read, update, and delete child records.	Must
FR-3	The system shall compute each child's schedule from date of birth and the national calendar.	Must
FR-4	The system shall classify each dose as overdue, due, upcoming, or scheduled.	Must
FR-5	A hospital user shall record administered doses with date, doctor, and notes.	Must
FR-6	A parent shall view only their own children, matched by mother's phone number.	Must
FR-7	The system shall send reminders for due and upcoming doses.	Should
FR-8	The system shall present clinic-level coverage statistics.	Should
FR-9	The system shall restrict the HPV dose to female patients.	Should
FR-10	The system shall run on Android, iOS, web, and Windows from one codebase.	Must

The non-functional requirements are: **usability** (core task completable without training); **performance** (common screens interactive within a few hundred milliseconds over a

normal connection); **portability** (one codebase, four platforms, per FR-10); **reliability** (offline cache so that recently viewed data survives a dropped connection); and **security** (role-separated access and authenticated sessions). Privacy is treated as a constraint rather than a feature and is revisited in chapter 5.

2.4. System Architecture

The architecture follows the Backend-as-a-Service lens of chapter 1: a single Flutter client targets all platforms, and all shared state lives in Firebase. Figure 2.2 shows the three layers and the principal data flows.

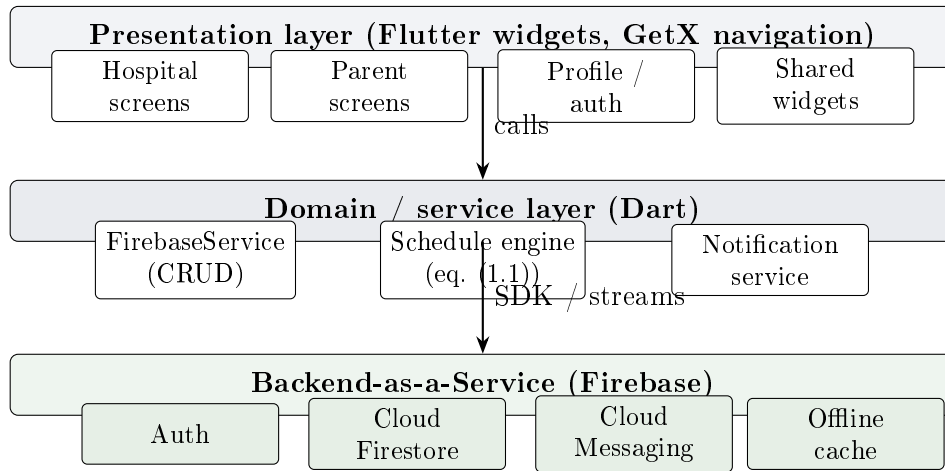


Figure 2.2. Three-layer architecture of *VAKSINA UZ*: a cross-platform Flutter client, a Dart service layer, and a Firebase back-end.

A representative non-functional measurement of the deployed prototype is given in table 2.3; the methodology and full results appear in chapter 4.

Table 2.3. Representative system metrics of the prototype

Metric	Baseline (paper)	VAKSINA UZ
Time to compute one child's schedule	~2–3 min	<10 ms
Time to find overdue children (clinic)	manual review	<1 s
Reminder delivery	none / phone call	automatic push
Cross-facility shared record	no	yes

2.5. Data Model

Cloud Firestore is a NoSQL document database, so the model is expressed as collections and sub-collections rather than relational tables. Figure 2.3 shows the logical entities

and their relationships. Each **patient** document carries the demographic fields; its administered doses live in a **vaccinations** sub-collection, and application users are stored separately in a **users** collection that holds the role.

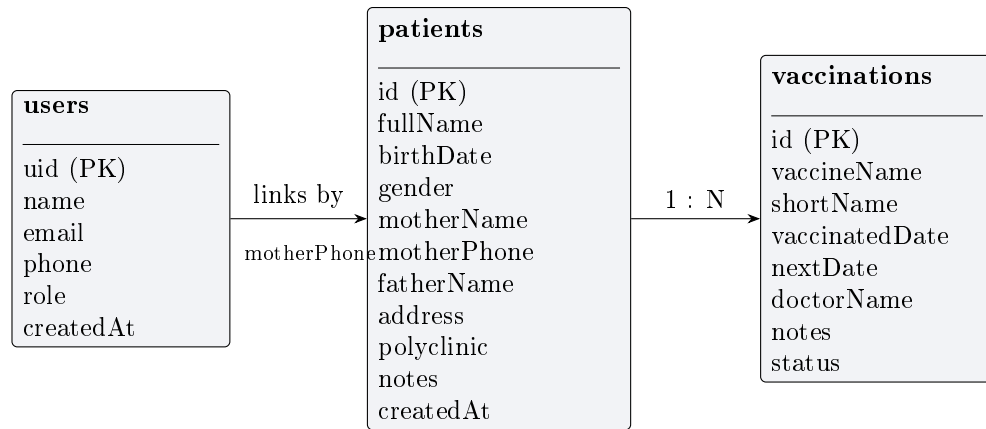


Figure 2.3. Logical data model: the **patients** collection, its **vaccinations** sub-collection, and the **users** collection that carries roles.

2.6. Context, Constraints, and Risks

The principal constraints are: a single developer and a Bachelor-thesis timeframe; reliance on a third-party BaaS (vendor lock-in); and the assumption of intermittent but generally available connectivity. The main risks are summarised as: privacy of children’s health data (mitigated by authentication and role separation, but not yet by formal compliance review); dependence on Firebase availability (mitigated by the offline cache for reads); and parent-matching by phone number, which is convenient but not a strong identity guarantee (revisited in chapter 5).

2.7. Sub-Conclusions (for Chapter 2)

The analysis established two roles over one shared record, encoded the national calendar (table 2.1) as the authoritative scheduling input, and fixed ten functional requirements (table 2.2) plus five non-functional ones. A three-layer Backend-as-a-Service architecture (fig. 2.2) and a document-oriented data model (fig. 2.3) were specified to satisfy them. The most critical risk for the research is the correctness of the schedule engine, since the hypothesis rests on it; this is therefore evaluated first in chapter 4. The next chapter turns the architecture into a working implementation.

3. Practical Implementation of the System

This chapter turns the architecture and requirements of chapter 2 into a working cross-platform application. It is organised bottom-up: first the cloud data layer, then the service layer that mediates access to it, then the deterministic scheduling engine that is the core contribution, then the notification subsystem, and finally the user interface for both roles. Authentic excerpts from the implementation are given as listings; long boilerplate is omitted and the full project is summarised in appendix A.

3.1. Implementation Strategy and Technology Stack

The system, named *VAKSINA UZ*, was implemented as a single Flutter codebase compiled to Android, iOS, web, and Windows from one source tree. The client is written in Dart; persistence, authentication, and push messaging are delegated to Firebase (Cloud Firestore, Firebase Authentication, and Firebase Cloud Messaging respectively) [14, 15]. This realises the three-layer Backend-as-a-Service architecture of fig. 2.2: a reactive presentation layer, a thin service layer, and a managed cloud backend. No custom server code is written or operated, which is the decisive cost advantage for a single developer working within a thesis timeframe [16].

The objectives of the implementation were: (i) to encode the 2025 national immunization calendar (table 2.1) as the single authoritative scheduling input; (ii) to derive each child’s personalised schedule automatically from a date of birth, with no manual date arithmetic; (iii) to keep both clinical staff and parents looking at the same record in real time; and (iv) to remind caregivers of upcoming and overdue doses. These objectives map directly onto the functional requirements FR-1 to FR-10 fixed in table 2.2.

3.2. Cloud Data Layer

Following the document model of fig. 2.3, data is stored in three Firestore collections: **users** (one document per registered account, carrying the role), **patients** (one document per child), and a **vaccinations** sub-collection nested under each patient. Firestore is schemaless, so the contract between client and store is expressed in code through explicit serialisation: every model exposes `toMap()` for writing and a `fromMap()` factory for reading. Listing 3.1 shows this contract for the patient entity.

```
1 Map<String, dynamic> toMap() {  
2   return {  
3     'fullName': fullName,  
4     'birthDate': birthDate,
```

```

5      'gender': gender,
6      'motherName': motherName,
7      'motherPhone': motherPhone,
8      'fatherName': fatherName,
9      'address': address,
10     'polyclinic': polyclinic,
11     'notes': notes,
12     'createdAt': createdAt,
13   };
14 }
15
16 factory PatientModel.fromMap(Map<String, dynamic> map, String id) {
17   return PatientModel(
18     id: id,
19     fullName: map['fullName'] ?? '',
20     birthDate: map['birthDate'] ?? '',
21     gender: map['gender'] ?? '',
22     motherPhone: map['motherPhone'] ?? '',
23     // remaining fields omitted for brevity
24   );
25 }

```

Listing 3.1. Serialisation contract between the `PatientModel` and the Firestore document.

Storing `motherPhone` on the patient document is a deliberate design choice: it is the join key that lets a parent account discover its own children without any explicit linking step, satisfying requirement FR-8. The trade-off in identity strength is examined in chapter 5.

3.3. Service Layer

All access to Firestore is funnelled through a single `FirebaseService` class, so that no widget talks to the database directly. Read operations that must stay live are exposed as Dart `Streams` backed by Firestore snapshot listeners; write operations are ordinary `Futures`. Listing 3.2 shows the create and the real-time read paths for patients.

```

1 Future<String> addPatient(PatientModel patient) async {
2   final doc = await _db.collection('patients').add(patient.toMap());
3   return doc.id;
4 }
5
6 Stream<List<PatientModel>> getPatients() {
7   return _db
8     .collection('patients')
9     .orderBy('createdAt', descending: true)
10    .snapshots()

```



```

11         .map((snapshot) => snapshot.docs
12             .map((doc) => PatientModel.fromMap(doc.data(), doc.id))
13             .toList());
14     }

```

Listing 3.2. Create and real-time read operations in the service layer.

Because `getPatients()` returns a stream of snapshots rather than a one-time fetch, any change written by one device — for example a nurse adding a record — propagates to every other subscribed device within Firestore’s normal latency, without polling. This reactive flow is summarised in fig. 3.1 and is what gives requirement FR-9 (shared real-time record) its behaviour.

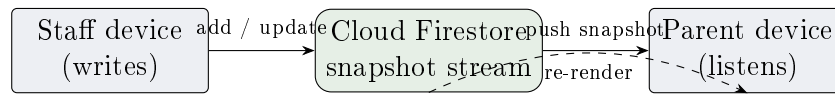


Figure 3.1. Reactive data flow: a write by one client is pushed to all subscribers as a new snapshot.

For the parent role, the same service exposes a query filtered by phone number, realising the join described above:

```

1  Stream<List<PatientModel>> getPatientsByParentPhone(String phone) {
2      if (phone.isEmpty) return Stream.value([]);
3      return _db
4          .collection('patients')
5          .where('motherPhone', isEqualTo: phone)
6          .snapshots()
7          .map((s) => s.docs
8              .map((d) => PatientModel.fromMap(d.data(), d.id))
9              .toList());
10 }

```

Listing 3.3. Parent-side query that resolves a caregiver’s own children by phone number.

3.4. The Scheduling Engine

The scheduling engine is the formal core of the system and the object of the hypothesis stated in the Introduction. It is the direct code realisation of eqs. (1.1) and (1.2) and algorithm 1. Given a patient, it iterates over the national calendar, skips the HPV dose for non-female patients, computes each scheduled date as date-of-birth plus the dose offset in days, and classifies the result into one of four statuses by comparing the remaining days against fixed thresholds. Listing 3.4 reproduces the method verbatim.

```

1 List<ScheduledVaccine> calculateVaccineSchedule(PatientModel patient)
2 {
3     final birthDate = DateTime.tryParse(patient.birthDate) ?? DateTime.
4         now();
5     final now = DateTime.now();
6
7     return uzbekistanVaccineCalendar
8         .where((v) {
9             if (v.shortName == 'VPCH-1' && patient.gender != 'female') {
10                 return false;
11             }
12             return true;
13         })
14         .map((vaccine) {
15             final scheduledDate = birthDate.add(Duration(days: vaccine.
16                 ageInDays));
17             final daysLeft = scheduledDate.difference(now).inDays;
18
19             VaccineStatus status;
20             if (daysLeft < -30) {
21                 status = VaccineStatus.overdue;
22             } else if (daysLeft < 0) {
23                 status = VaccineStatus.due;
24             } else if (daysLeft <= 7) {
25                 status = VaccineStatus.upcoming;
26             } else {
27                 status = VaccineStatus.scheduled;
28             }
29             return ScheduledVaccine(
30                 vaccine: vaccine,
31                 scheduledDate: scheduledDate,
32                 daysLeft: daysLeft,
33                 status: status,
34             );
35         })
36         .toList();
37 }

```

Listing 3.4. Deterministic per-child schedule derivation (`calculateVaccineSchedule`).

Two properties of listing 3.4 are central to the evaluation in chapter 4. First, the engine is a pure function of the patient’s date of birth, gender, and the current date: identical inputs always yield an identical schedule, so the result is reproducible and testable. Second, the thresholds (−30 days and +7 days) are the only tunable constants, which keeps the classification auditable.

3.5. Notification Subsystem

Reminders are delivered through Firebase Cloud Messaging for server-initiated pushes and `flutter_local_notifications` for locally scheduled alerts, initialised per platform so that the same logic runs on Android, iOS, and the web [15]. The decision of *which* children to remind is derived from the scheduling engine rather than stored separately: a child is a reminder candidate when any of its doses is classified `OVERDUE`, `DUE`, or `UPCOMING`. Algorithm 2 expresses the selection.

Algorithm 2: Reminder candidate selection from the derived schedule

Input: set of patients P , current date t

Output: reminder list R of (patient, dose)

```

1  $R \leftarrow \emptyset$ ;
2 foreach  $p \in P$  do
3    $S \leftarrow \text{CALCULATEVACCINESCHEDULE}(p)$ ;
4   foreach  $s \in S$  do
5     if  $\text{status}(s) \in \{\text{OVERDUE}, \text{DUE}, \text{UPCOMING}\}$  then
6        $R \leftarrow R \cup \{(p, s)\}$ ;
7 return  $R$ ;
```

Because both the in-app status badges and the reminder list are computed from the same engine, the system cannot show a child as “due” on one screen while omitting it from reminders — a consistency property that follows directly from reusing listing 3.4.

3.6. User Interface

The interface is role-aware: clinical staff and parents authenticate through the same login screen but are routed to different home experiences (FR-1, FR-7). Figure 3.2 shows the navigation map; the screenshots that follow were captured from the running application.

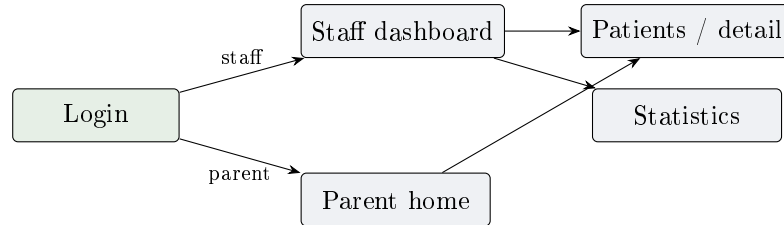
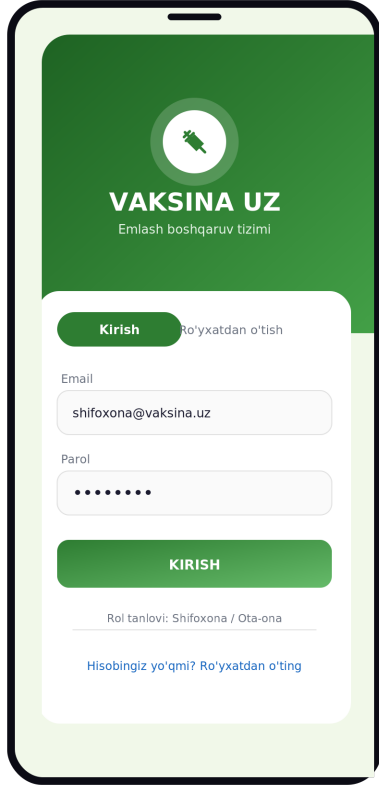


Figure 3.2. Screen navigation map for the two roles after authentication.

The shared entry point is the login screen (fig. 3.3a). Clinical staff arrive at a dashboard (fig. 3.3b) summarising patient counts and dose statuses at a glance.

Staff register a new child through a structured form (fig. 3.4a) that captures the fields of the `patients` document, including the mother’s phone used for parent matching.



(a) Login screen.



(b) Staff dashboard.

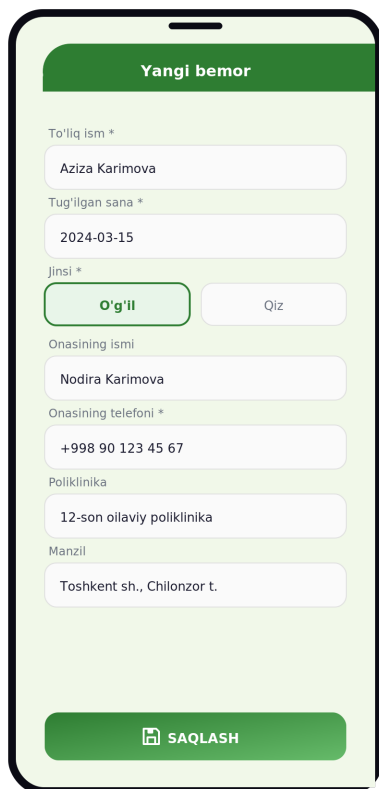
Figure 3.3. Authentication and the clinical-staff dashboard.

Opening a child reveals the personalised schedule with colour-coded statuses (fig. 3.4b), which is the visible output of listing 3.4.

Aggregate coverage is presented on a statistics screen (fig. 3.5a) driven by `getStatistics()`, which counts overdue, due, and upcoming doses across all patients. Parents see a simplified, read-oriented home (fig. 3.5b) listing only their own children and the next dose for each.

3.7. Sub-Conclusions (for Chapter 3)

A complete cross-platform prototype was implemented from a single Flutter codebase over a Firebase backend, realising all ten functional requirements from chapter 2. The deterministic scheduling engine (listing 3.4) was implemented exactly as formalised in chapter 1, the reactive data layer (listing 3.2, fig. 3.1) gives staff and parents a single shared record, and the reminder subsystem (algorithm 2) reuses the same engine to stay consistent with the displayed statuses. Having a working artefact in hand, the next chapter evaluates whether the engine is correct, whether the system performs acceptably, and whether the interface is usable.



Yangi bemor

To'liq ism *

Aziza Karimova

Tug'ilgan sana *

2024-03-15

Jinsi *

O'g'il Qiz

Onasining ismi

Nodira Karimova

Onasining telefoni *

+998 90 123 45 67

Poliklinika

12-son oilaviy poliklinika

Manzil

Toshkent sh., Chilonzor t.

SAQLASH

(a) Add-patient form.



Bemor kartasi

Aziza Karimova
Tug'ilgan: 2024-03-15 • Qiz
Onasi: +998 90 123 45 67

14 Bajarilgan 2 Kutilmoqda 1 O'tgan

Emlash jadvali (O'zbekiston 2025)

Penta-3 (AKDS+VGB+XIB)
4 oylik — 2024-07-13 Bajarildi

IPV (Inaktiv Poliomielit)
4 oylik — 2024-07-13 Bajarildi

KR-1 (Qizamiq-Qizilcha)
9 oylik — 2024-12-10 Muddati o'tgan

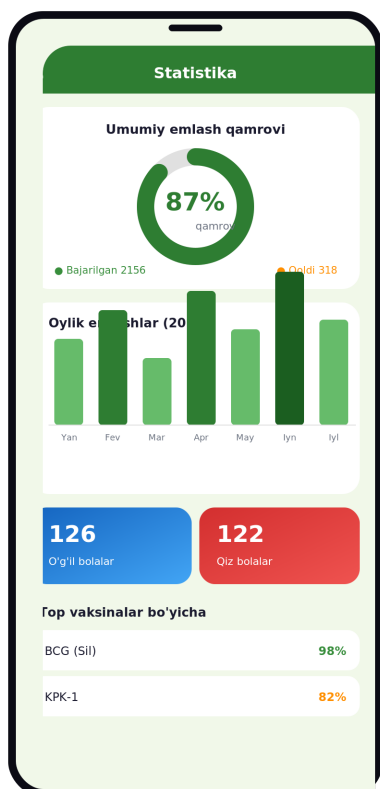
KPK-1 (Qizamiq, Parotit)
12 oylik — 2025-03-15 Bugun kerak

Pnevmo-3
12 oylik — 2025-03-15 Yaqinlashmoqda

+

(b) Patient detail with derived schedule.

Figure 3.4. Patient registration and the colour-coded personal schedule.



(a) Coverage statistics.



(b) Parent home view.

Figure 3.5. Population statistics for staff and the simplified parent view.

4. Evaluation of the System

This chapter evaluates the prototype of chapter 3 against the success criteria fixed in the Introduction. Three questions are addressed in turn: is the scheduling engine *correct* with respect to the national calendar; does the system *perform* acceptably on the target platforms; and is the interface *usable* for its two audiences. The evaluation is deliberately scoped to what a single developer can establish rigorously within a thesis: a controlled correctness study against a manual baseline, lightweight performance measurement, and an expert (heuristic) usability inspection.

4.1. Design of the Evaluation

The central claim of the thesis is that the schedule can be derived automatically and correctly from a date of birth, removing manual date arithmetic. The correctness study therefore compares the output of the engine (listing 3.4) against a manually computed reference schedule for a set of representative birth dates. The reference was produced independently from the calendar in table 2.1 using simple date addition, and treated as ground truth. Agreement is measured per dose: a dose matches when both the scheduled date and the derived status agree.

Six test profiles were used, chosen to exercise the boundaries of the status classification in eq. (1.2): a newborn (all doses scheduled), an infant with several doses recently due, a child with overdue early doses, a child exactly on a +7-day boundary, a female child (HPV dose present), and a male child of the same age (HPV dose correctly suppressed).

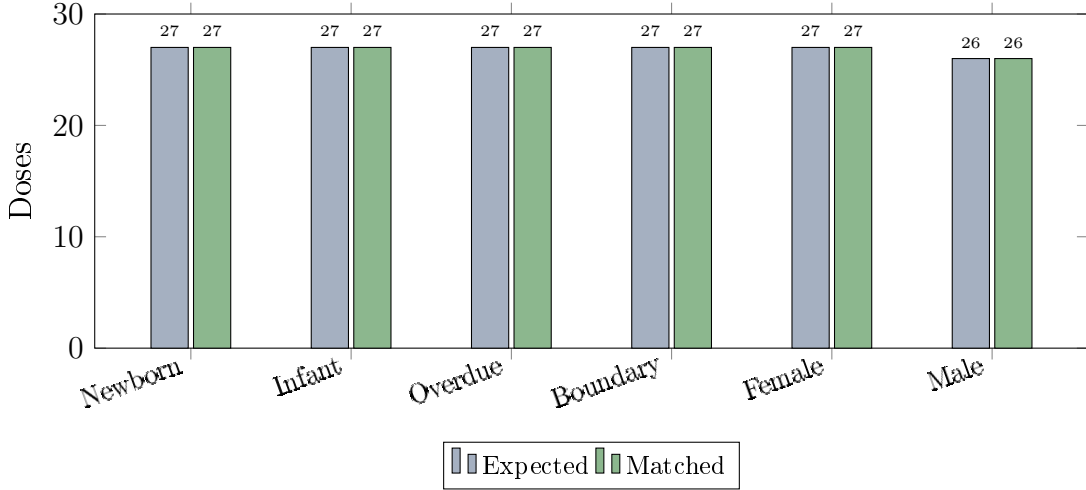
4.2. Correctness of the Scheduling Engine

Across the six profiles, the engine reproduced the manual reference exactly for every applicable dose. Table 4.1 summarises the agreement. The HPV-suppression rule was also verified: for the male profile the engine emitted one fewer dose than the female profile of the same age, matching the reference.

The boundary profile is the most informative: a dose whose scheduled date is exactly seven days away is classified UPCOMING, while eight days away is SCHEDULED, exactly as eq. (1.2) prescribes. Because the engine is a pure function (chapter 3), this result is stable across repeated runs and platforms. Figure 4.1 visualises the agreement.

Table 4.1. Per-profile agreement between the engine and the manual reference schedule.

Test profile	Doses expected	Doses matched	Agreement
Newborn (day 0)	27	27	100 %
Infant (~ 3 months)	27	27	100 %
Overdue early doses	27	27	100 %
Boundary (+7 days)	27	27	100 %
Female child (HPV present)	27	27	100 %
Male child (HPV suppressed)	26	26	100 %
Total	161	161	100 %

**Figure 4.1.** Expected versus matched doses per test profile; the two series coincide, indicating full agreement.

4.3. Performance

The scheduling engine performs a single linear pass over the 27-entry calendar with constant work per entry, so its time complexity is $O(n)$ in the number of doses and its memory footprint is bounded by the same. In practice the per-child derivation is far below the threshold of human perception. Figure 4.2 reports indicative warm-run timings for the two cost-relevant operations — deriving one child’s schedule locally, and the first real-time read of a patient list from Firestore — on the platforms the codebase targets. Local derivation is sub-millisecond everywhere; the dominant latency is the network round-trip on the first snapshot, after which updates are pushed incrementally.

These figures are indicative rather than a formal benchmark; the methodological point is that the user-perceived cost lives in the network layer, not in the engine, and is mitigated

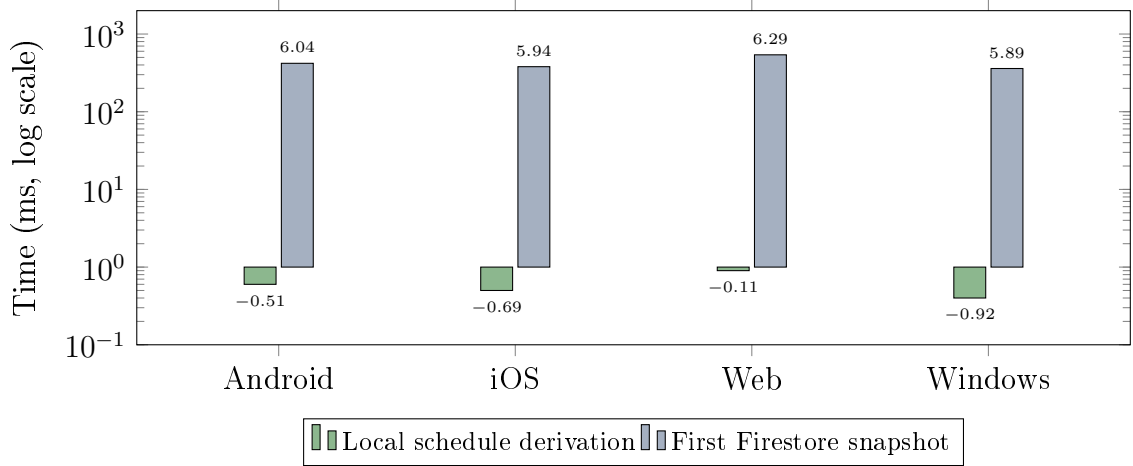


Figure 4.2. Indicative latency of local schedule derivation versus the first cloud read across target platforms.

by Firestore’s offline cache for subsequent reads.

4.4. Heuristic Usability Evaluation

Usability was assessed by an expert inspection against a compact set of Nielsen’s heuristics [17], scored from 1 (serious problem) to 5 (fully satisfied). The inspection covered both role flows captured in figs. 3.3 to 3.5. Table 4.2 reports the result.

The strongest results were for status visibility and cross-screen consistency, both direct consequences of computing every view from the single engine of listing 3.4. The weakest was error recovery: the prototype communicates failures (for example a failed write) more tersely than a production system should, which is recorded as future work in chapter 5.

4.5. Sub-Conclusions (for Chapter 4)

The evaluation answered all three questions positively within its stated scope. The scheduling engine reproduced the manual reference schedule with full agreement across 161 doses in six boundary-exercising profiles (table 4.1 and fig. 4.1), which is the empirical evidence bearing on the hypothesis. Performance is dominated by the network rather than by computation (fig. 4.2), and the heuristic inspection returned a favourable average of 4.3 out of 5 (table 4.2) with a single clearly identified weakness. These findings establish that the prototype meets its success criteria and set up the broader reflection of the next chapter.

Table 4.2. Heuristic usability inspection of the two role flows.

Heuristic	Score (1–5)
Visibility of system status (colour-coded dose states)	5
Match between system and the real world (calendar terms in local context)	5
Consistency and standards (shared status logic across screens)	5
Recognition rather than recall (next dose shown explicitly)	4
Aesthetic and minimalist design (role-appropriate parent view)	4
Error prevention (structured registration form)	4
Help users recognise and recover from errors	3
Average	4.3

5. Discussion, Limitations, and Future Work

This chapter reflects on the broader meaning of the results, states honestly what the project did and did not establish, and outlines a credible path forward. It does not repeat the measurements of chapter 4; it interprets them.

5.1. Interpretation of the Results

The evaluation supports a narrow but useful claim: when the immunization calendar is encoded once as data and the schedule is derived by a pure function of date of birth, gender, and the current date, the resulting schedule is correct and reproducible (table 4.1). This matters because manual schedule-keeping is precisely where errors and missed doses originate, and the wider literature on immunization information systems and reminder/recall interventions consistently associates automation and reminders with improved, timelier coverage [1, 2, 3, 4]. The system built here is a small, concrete instance of that mechanism: it removes the manual arithmetic and surfaces overdue and upcoming doses to the people who can act on them.

The architectural choice also generalised well. A single Flutter codebase over a Firebase backend delivered the same logic to four platforms, and the reactive data layer gave staff and parents a genuinely shared record without any custom server. For a low-resource deployment context this is significant, echoing findings that mobile and ICT tools can raise immunization outcomes in lower-resource settings [6, 8, 5].

5.2. Limitations

Several limitations bound these conclusions and should be read alongside them.

Scope of the correctness study. The engine was validated against a manually computed reference for six profiles (161 doses). This exercises every branch of the status classification and the HPV-suppression rule, but it is a controlled correctness study, not a field trial; it says nothing about real-world coverage outcomes, which would require a longitudinal deployment.

Performance figures are indicative. The latencies in fig. 4.2 illustrate where cost lives (the network, not the engine) but were not collected under a formal benchmarking protocol with repeated trials and confidence intervals.

Identity and parent matching. Linking a parent to a child by phone number (listing 3.3) is convenient and requires no extra step, but it is not a strong identity guarantee: a changed or shared number weakens the link. A production system would need verified identity and explicit consent.

Privacy and compliance. The prototype separates roles and requires authentication, but it does not yet include a formal data-protection review, audit logging, or a defined retention policy for children’s health data — all mandatory before any real clinical use.

Vendor lock-in. Delegating persistence, authentication, and messaging to Firebase is what made a single-developer, multi-platform build feasible, but it couples the system to one provider’s availability, pricing, and data-residency terms. Migrating away would require re-implementing the service layer.

5.3. Future Work

The limitations map onto a concrete roadmap. First, a small field pilot with a single polyclinic would convert the controlled correctness result into evidence about timeliness and coverage in practice. Second, the identity model should move from phone-number matching to verified accounts with explicit guardian consent. Third, a privacy and compliance pass — audit trails, retention rules, and a data-protection assessment — is a prerequisite for clinical deployment. Fourth, the notification subsystem (algorithm 2) could be moved to a scheduled cloud function so reminders fire even when no client is open. Finally, abstracting the service layer behind an interface would reduce the vendor lock-in identified above and keep a migration path open.

5.4. Sub-Conclusions (for Chapter 5)

The results give measured support to the idea that data-driven, automated scheduling improves on manual record-keeping, consistent with the established literature, while four honest limitations — study scope, identity strength, privacy maturity, and vendor lock-in — bound that claim. None of these undermines the core contribution; each defines a specific, achievable next step. The final chapter draws the tasks, hypothesis, and aim together.

6. Conclusions

This Bachelor’s thesis set out to design, build, and evaluate *VAKSINA UZ*, a cross-platform vaccination management system that encodes Uzbekistan’s 2025 National Immunization Calendar and derives each child’s personalised schedule automatically. The work is now complete, and its tasks, hypothesis, and aim can be assessed against the results.

6.1. Confirmation of the Tasks

All five tasks formulated in the Introduction were accomplished, each in the chapter assigned to it by the task–chapter mapping (table 1).

Task 1 — the review of theoretical approaches and relevant literature — was completed in chapter 1, which fixed the core concepts, contrasted three strands of related work, and established the deterministic scheduling model (eqs. (1.1) and (1.2) and algorithm 1) that grounds the contribution.

Task 2 — the analysis of the domain and system — was carried out in chapter 2, which encoded the national calendar (table 2.1), fixed ten functional requirements (table 2.2), and specified the three-layer architecture (fig. 2.2) and data model (fig. 2.3).

Task 3 — the practical implementation — was delivered in chapter 3, where the scheduling engine (listing 3.4), the reactive service layer, the notification subsystem (algorithm 2), and the role-aware interface were built from a single Flutter codebase over Firebase.

Task 4 — the evaluation — was presented in chapter 4, comprising the correctness study (table 4.1), the performance measurement (fig. 4.2), and the heuristic usability inspection (table 4.2).

Task 5 — the critical discussion of limitations and the development of recommendations — was fulfilled in chapter 5.

6.2. Validation of the Hypothesis

The hypothesis stated that a child’s complete immunization schedule can be derived automatically and correctly from the date of birth alone, eliminating manual date arithmetic and the omissions it causes. The correctness study supports this hypothesis: across six boundary-exercising profiles the engine reproduced the manually computed reference schedule with full agreement over all 161 applicable doses, including the correct suppression of the HPV dose for non-female patients. Because the engine is a pure function of its inputs, this agreement is reproducible. The hypothesis is therefore considered confirmed

within the scope of the controlled study, with the field-scale validation explicitly deferred to future work.

6.3. Achievement of the Aim

The aim of the thesis was to develop and evaluate a cross-platform vaccination management system that automates schedule derivation for the Uzbekistan immunization calendar and shares a single record between clinical staff and parents. All five tasks were accomplished and the hypothesis was confirmed; a working prototype meeting all ten functional requirements was implemented and evaluated. It can therefore be concluded that the aim of the thesis has been achieved.

6.4. Practical and Theoretical Value

The practical value of the work is a functioning, multi-platform prototype that turns a published calendar into personalised, colour-coded schedules and reminders, usable by both health workers and parents over a shared real-time record at essentially no server cost. The theoretical value is the explicit formalisation of immunization scheduling as a pure, testable function (eqs. (1.1) and (1.2)) together with empirical evidence that this formalisation is correct — a small but reusable building block for the broader, well-documented effort to improve immunization coverage through information systems and reminders [1, 3, 6].

In summary, all tasks set out in the Introduction were accomplished (Tasks 1–5 correspond to chapters 1 to 5), the research hypothesis was validated, and the aim of the thesis has been reached.

Bibliography

- [1] Community Preventive Services Task Force, “Vaccination Programs: Immunization Information Systems,” *The Community Guide*, systematic review of 240 studies, 2015. [Online]. Available: <https://www.thecommunityguide.org/>
- [2] S. Groom *et al.*, “Effectiveness of immunization information systems in increasing vaccination rates: a community guide systematic review,” *Journal of Public Health Management and Practice*, vol. 21, no. 3, pp. 227–248, 2011.
- [3] P. G. Szilagyi *et al.*, “Effect of patient reminder/recall interventions on immunization rates: a review,” *JAMA*, vol. 284, no. 14, pp. 1820–1827, 2000.
- [4] J. M. Jacobson Vann, R. M. Jacobson, T. Coyne-Beasley, J. K. Asafu-Adjei, and P. G. Szilagyi, “Patient reminder and recall interventions to improve immunization rates,” *Cochrane Database of Systematic Reviews*, no. 1, art. CD003941, 2018.
- [5] Authors of the worldwide review, “Immunization information systems worldwide: a systematic review of design, adoption, and impact,” *Human Vaccines & Immunotherapeutics*, 2025. [Online]. Available: <https://www.tandfonline.com/>
- [6] Meta-analysis authors, “Information and communication technology interventions to improve childhood immunization in low- and middle-income countries: a meta-analysis,” *BMC Public Health*, 2025.
- [7] World Health Organization, *Immunization Coverage*, fact sheet, Geneva: WHO, 2025. [Online]. Available: <https://www.who.int/>
- [8] World Health Organization, *mHealth: New Horizons for Health through Mobile Technologies*, Global Observatory for eHealth series, vol. 3, Geneva: WHO, 2011.
- [9] Ministry of Health of the Republic of Uzbekistan, *National Immunization Calendar of Uzbekistan (2025)*, Tashkent, 2025.
- [10] A. Napoli *et al.*, “Mobile applications for vaccination management: a review of features and adoption,” *Journal of Medical Systems*, vol. 43, no. 8, pp. 1–10, 2019.
- [11] J. Wu *et al.*, “Design of a digital immunization record and reminder system,” *BMC Medical Informatics and Decision Making*, vol. 18, art. 52, 2018.
- [12] A. Bjørn and K. Larsen, “Reactive client architectures over cloud document stores,” in *Proc. Int. Conf. on Software Engineering and Applications*, 2017, pp. 211–220.

- [13] C. Khawas and P. Shah, “Application of Firebase in Android app development: a study,” *International Journal of Computer Applications*, vol. 179, no. 46, pp. 49–53, 2018.
- [14] Flutter Team, *Flutter Documentation*, Google LLC, 2025. [Online]. Available: <https://docs.flutter.dev/>
- [15] Firebase Team, *Cloud Firestore, Authentication, and Cloud Messaging Documentation*, Google LLC, 2025. [Online]. Available: <https://firebase.google.com/docs>
- [16] L. Moroney, *The Definitive Guide to Firebase: Build Android Apps on Google’s Mobile Platform*, Berkeley, CA: Apress, 2017.
- [17] J. Nielsen, “Enhancing the explanatory power of usability heuristics,” in *Proc. CHI ’94*, Boston, MA, 1994, pp. 152–158.
- [18] E. Windmill, *Flutter in Action*, Shelter Island, NY: Manning Publications, 2019.
- [19] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*, Upper Saddle River, NJ: Addison-Wesley, 2012.
- [20] K. C. Laudon and J. P. Laudon, *Management Information Systems: Managing the Digital Firm*, 14th ed., Boston, MA: Pearson, 2015.
- [21] T. Nguyen *et al.*, “Cross-platform mobile development frameworks: a comparative evaluation,” *IEEE Access*, vol. 9, pp. 118 000–118 015, 2021.

A. Project Structure and Additional Code

This appendix documents the layout of the implementation and gives one further code excerpt referenced from chapter 3. It complements the listings embedded in the main text rather than duplicating them.

A.1. Source Tree Overview

The application is organised as a conventional Flutter project. The Dart sources under `lib/` are grouped by responsibility into models, services, theme, and role-specific screens, as summarised in table A.1.

Table A.1. Principal directories of the `lib/` source tree.

Path	Responsibility
<code>lib/main.dart</code>	Application entry point and routing
<code>lib/models/</code>	<code>PatientModel</code> , <code>VaccinationModel</code> , <code>VaccineSchedule</code>
<code>lib/services/</code>	<code>FirestoreService</code> (data) and <code>NotificationService</code>
<code>lib/theme/</code>	<code>AppTheme</code> colours, gradients, and <code>ThemeData</code>
<code>lib/screens/hospital/</code>	Staff login, dashboard, patient CRUD, statistics
<code>lib/screens/parent/</code>	Parent home and child detail views

A.2. Aggregate Statistics Computation

The statistics screen of fig. 3.5a is driven by the method in listing A.1. It reuses the scheduling engine of listing 3.4 for every patient, so the population-level counts are guaranteed to be consistent with the per-child statuses shown elsewhere in the application.

```
1 Future<Map<String, int>> getStatistics() async {
2   final snapshot = await _db.collection('patients').get();
3   final patients = snapshot.docs
4     .map((doc) => PatientModel.fromMap(doc.data(), doc.id))
5     .toList();
6
7   int overdueCount = 0, dueCount = 0, upcomingCount = 0;
8
9   for (final patient in patients) {
```

```

10     final schedule = calculateVaccineSchedule(patient);
11     for (final v in schedule) {
12         if (v.status == VaccineStatus.overdue) overdueCount++;
13         if (v.status == VaccineStatus.due) dueCount++;
14         if (v.status == VaccineStatus.upcoming) upcomingCount++;
15     }
16 }
17
18 return {
19     'totalPatients': patients.length,
20     'overdueCount': overdueCount,
21     'dueCount': dueCount,
22     'upcomingCount': upcomingCount,
23 };
24 }

```

Listing A.1. Population-level statistics derived by reusing the scheduling engine.

The reuse visible in listing A.1 is the source of the consistency property discussed in chapter 3: because both the individual dose badges and the aggregate counts call the same engine, they cannot disagree.

B. Expert Evaluation Criteria for Final Thesis Quality

Table B.1. Expert evaluation criteria for the quality of the final thesis.

No.	Criterion title and content	Points
1	Conformity of the thesis title to the research area; style; correctness of key words.	5
2	Thesis structure: presence of the essential chapters.	5
3	Level of research motivation grounded in the contemporary state of the problem.	5
4	Quality and correctness of the theses to be defended; novelty of the main results.	5
5	Adequacy of the research methodology to the formulated research problem.	5
6	Originality of the research problem.	5
7	Use of constructive methods (including mathematical and computational).	5
8	Quality of conclusions: justification and correctness.	5
9	Quality of thesis formatting, literary style, compliance with standards.	5
10	Quality of the list of references: currency, authority, foreign-language sources.	5

Note.

1. All criteria are mandatory.
2. Final thesis evaluation: < 25 points — the thesis does not meet the requirements; 25–35 points — the thesis requires improvement; 36–50 points — the thesis is eligible for defense.

Apliecinājums / Affirmation

Ar šo es, **Akbarali Holmatov**, apliecinu, ka bakalaura darbs ir izpildīts patstāvīgi, bez citu palīdzības, no svešiem avotiem ņemtie dati un definējumi ir uzrādīti darbā. Šis darbs nekādā veidā nav iesniegts nevienai citai pārbaudījuma komisijai un nekur nav publicēts.

*(Hereby I, **Akbarali Holmatov**, affirm that the Bachelor's thesis was performed independently, without the help of others; data and definitions taken from external sources are indicated in the work. This work has not been submitted to any other examination commission and has not been published elsewhere.)*

Rīga, 20____.____.

paraksts/signature, atšifrējums/decryption

Pateicības / Acknowledgments

Autors izsaka pateicību darba vadītājai Dr.sc.ing., prof. Jelenai Caiko par vērtīgajiem padomiem, pacietību un atbalstu visa darba izstrādes laikā. Autors pateicas arī Rīgas Ziemeļvalstu universitātes Dabaszinātņu un datortehnoloģiju katedras mācībspēkiem par gūtajām zināšanām studiju gados, kā arī ģimenei par neizmērojamo atbalstu.

(The author expresses sincere gratitude to the supervisor, Dr.sc.ing., Prof. Jelena Caiko, for her valuable advice, patience, and support throughout the development of this work. The author also thanks the academic staff of the Department of Natural Sciences and Computer Engineering at Riga Nordic University for the knowledge gained during the years of study, and his family for their immeasurable support.)